

Rules Refine the Riddle: Global Explanation for Deep Learning-Based Anomaly Detection in Security Applications

Dongqi Han
Tsinghua University
Zhongguancun Laboratory
Beijing, China
handq19@mails.tsinghua.edu.cn

Zhiliang Wang
Tsinghua University
Zhongguancun Laboratory
Beijing, China
wzl@cernet.edu.cn

Ruitao Feng
Singapore Management University
Singapore, Singapore
rtfeng@smu.edu.sg

Minghui Jin
State Grid Shanghai Municipal
Electric Power Company
Shanghai, China
jmh2006@qq.com

Wenqi Chen
Tsinghua University
Zhongguancun Laboratory
Beijing, China
chenwq19@mails.tsinghua.edu.cn

Kai Wang
Tsinghua University
Zhongguancun Laboratory
Beijing, China
k-wang20@mails.tsinghua.edu.cn

Su Wang
Zhongguancun Laboratory
Beijing, China
wangsu@zgclab.edu.cn

Jiahai Yang
Tsinghua University
Zhongguancun Laboratory
Beijing, China
yang@cernet.edu.cn

Xingang Shi
Tsinghua University
Zhongguancun Laboratory
Beijing, China
shixg@cernet.edu.cn

Xia Yin
Tsinghua University
Zhongguancun Laboratory
Beijing, China
yxia@tsinghua.edu.cn

Yang Liu
Nanyang Technological University
Singapore, Singapore
yangliu@ntu.edu.sg

Abstract

Deep learning (DL) based anomaly detection has shown great promise in the field of security due to its remarkable performance in various tasks. However, the issue of poor interpretability in DL models has significantly impeded their deployment in practical security applications. Despite the progress made in existing studies on DL explanations, the majority of them focus on providing local explanations for individual samples, neglecting the global understanding of the model knowledge. Furthermore, most explanations for supervised models fail to apply to anomaly detection due to their different learning mechanisms.

In this work, we address the gap in the existing research by proposing **GEAD**, a novel global explanation for DL-based anomaly detection, to extract high-fidelity rules from DL models. We apply **GEAD** to two security applications, network intrusion detection and system log anomaly detection, and demonstrate the efficacy with three usages: comparing model knowledge with expert knowledge, identifying knowledge discrepancies between models, and combining model and expert knowledge. We provide several case studies to showcase how **GEAD** can significantly enhance existing anomaly detection systems. Moreover, we provide a real-world deployment in a SCADA system to showcase the potential in practice. Some important insights are drawn to help the community understand and improve anomaly detection systems in security.

CCS Concepts

• **Computing methodologies** → **Anomaly detection**; • **Security and privacy** → **Intrusion detection systems**; *File system security*.

Keywords

Deep Learning Interpretability; Anomaly Detection; Deep Learning-based Security Applications; Rule Extraction

CCS '24, October 14–18, 2024, Salt Lake City, UT, USA.

1 Introduction

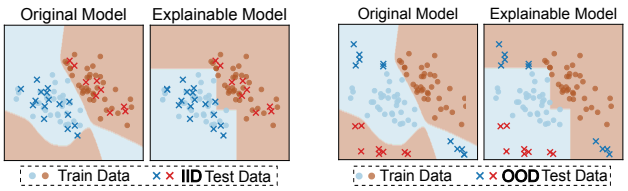
Anomaly detection techniques aim to identify unusual patterns or behaviors that deviate from normal system operations, signaling

potential security breaches [10]. With the rapid growth of interconnected devices and the increasing sophistication of cyber threats, anomaly detection has become an essential component in the security domain [2]. In recent years, deep learning (DL) based anomaly detection methods have demonstrated promising advantages and advancements in several security-related tasks, such as detecting network intrusions [45, 60], system abnormal states [16, 43], host-based threats [62, 74], and lateral movement in advanced persistent threats [7, 35]. Compared with traditional methods, the advantages of DL-based anomaly detection include the ability to capture complex patterns from large-scale and high-dimensional data [61], as well as detect unforeseen threats through *zero-positive* learning [15, 27, 28], i.e., learning with normal data alone.

Despite the remarkable progress, one practical concern of DL models lies in their black-box nature, where the internal working and model mechanism are often opaque and difficult to explain [9]. Security analysts require insights into the decision-making process to ensure that the anomaly detection system is correctly identifying genuine threats [28]. The lack of interpretability significantly impedes the deployment of DL-based methods in practical security applications. It becomes challenging to determine whether the model has captured the *intended* or *reasonable* knowledge, leaving security professionals uncertain about the effectiveness of the anomaly detection models.

The issue of interpretability and explanation of deep neural networks (DNN) has gained significant attention recently and researchers have explored various techniques, particularly in the domains of text and image [24]. In the security domain, some efforts have been made to address interpretability. However, they have predominantly focused on *local explanations* [28, 64] and supervised learning approaches [32, 44]. While local explanations offer valuable insights into specific samples or predictions by finding influential parts, they fall short of providing a comprehensive understanding of the *knowledge* learned by the model. Although there are a few global explanations for supervised learning, our first key insight in this study is that **prior global explanations for supervised models are not suitable for anomaly detection**, as they rely on prior knowledge and labeled training data of anomalies that are never used in zero-positive learning. Therefore, the intrinsic detection logic of the two kinds of learning is significantly different. Thus, the corresponding explanation for anomaly detection should be carefully reconsidered, as validated in §4.

The basic idea of the existing global explanation of DNN is to employ an *explainable* model (such as decision trees) to substitute the *original* neural network. The aim is to achieve maximal consistency in predictions between the explainable and original models, called "*fidelity*". However, existing methods tend to use a test set that is independent and identically distributed (IID) with the training samples for the explainable model to evaluate fidelity. As shown in Fig. 1, our second key insight is that **evaluating with IID test samples may engender a false sense of fidelity**. After training, the explainable model may have a completely or partially different decision-making mechanism (boundary) from the original model. However, IID test samples do not readily expose such discrepancy; it becomes obvious only when facing out-of-distribution (OOD) test samples.



(a) Test w/ IID ("Fidelity"=100%) **(b) Test w/ OOD (Fidelity=0%)**

Figure 1: Evaluate with IID cannot reflect real fidelity. In this toy example, the original and explainable models have different decision boundaries (background colors). Evaluating fidelity with (a) IID data does not reflect this difference (i.e., two models have the same decisions for IID data on the left figure), whereas using (b) OOD data does (significant decision differences on the right).

In light of the above insights, we propose **GEAD**, a novel global explanation method for DL-based anomaly detection models. We leverage the tree-based learning model as the explainable model since it consists of *rules*, which are widely employed in security domains [1] and are considered user-friendly and intuitive for security practitioners. Different from most existing tree-based explanations that directly utilize the binary classification outcomes (normal/abnormal), we firstly train a *regression* tree to approximate the output *probabilities* (i.e., anomaly scores) yielded by the original DL-based anomaly detection models. This increases the fidelity by enabling the explainable model to capture the abnormal degree and distribution learned by the original model, not only the binary results. Different from Trustee [32], we then refine the base regression tree with OOD samples to achieve real fidelity (recall Fig. 1) through locating *low-confidence (LC) feature spaces* . LC spaces refer to specific leaves in the regression tree with a low level of decision-making confidence. In other words, LC spaces are more likely to include OOD samples causing discrepancies between the explainable and original models. Therefore, we explore these feature spaces through well-designed data augmentation methods to generate fine-grained decision rules to enhance fidelity.

We conduct extensive experiments to demonstrate the effectiveness of **GEAD** and its potential to improve DL-based anomaly detection over two typical security applications, network intrusion detection and log anomaly detection, with four public datasets. After that, we propose a guideline of usages and several case studies for **GEAD**, which tell what analysis can be performed on anomaly detection systems and showcase how to improve them. Several important insights are concluded for the community. Real-world evaluation on a grid supervisory control and data acquisition (SCADA) system is also conducted to demonstrate the potential of **GEAD** in improving practical anomaly detection systems.

Contributions. This study makes the following contributions:

- By revealing the shortcomings of the IID-based approach in prior work (§1), we propose **GEAD**, a novel global explanation for DL-based anomaly detection models (§3), and provide implementations of **GEAD** on two types of DL-based anomaly detection[1].

[1]The implementation of **GEAD** is available at: https://github.com/dongtsi/GEAD

- By presenting more feasible metrics on fidelity (§4.1), we conduct several experiments on two security applications to demonstrate the effectiveness of **GEAD** (§4.2) and showcase the usage of **GEAD** to improve the underlying systems (§4.3). A real-world evaluation is also conducted to show the potential of **GEAD** in practice (§5).
- By analyzing existing anomaly detection systems through **GEAD**, we conclude important insights to help the community understand and improve anomaly detection systems in security (§4.4).

2 Background and Related Work

2.1 Deep Learning-based Anomaly Detection

According to the mechanism of training and detection, DL-based anomaly detection models can be divided into two types:

Reconstruction-based. This type of anomaly detection model consists of generative deep learning models such as Autoencoders (AE) and Generative Adversarial Networks (GAN). [45, 66, 73, 76]. During their training phase, the objective is to reduce the discrepancy between the normal data inputs and the corresponding outputs of the anomaly detection models, i.e., minimizing the reconstruction, such as the mean square error (MSE). They then identify anomalies by calculating the reconstruction error between the model's inputs and the outputs and then comparing it with an anomaly detection threshold. One example is a DL-based network intrusion detection system called Kistune [45], which extracts statistical features from network packages and proposes a two-layer ensemble of AE called KitNET as the anomaly detection model.

Prediction-based. This type of anomaly detection model is used for time-series or sequential data and consists of sequential models such as Recurrent Neural Networks (RNNs) and Long Short-Term Memory (LSTM) networks, as documented in references like [15, 16, 69]. For a normal time series, which is a sequence of time points, prediction-based approaches train the RNN to get better at forecasting the next time point in the sequence based on the previous ones. Like reconstruction-based models, these RNNs are designed to predict the next data point in a normal time series with a higher probability. However, when faced with anomalies, the model is prone to making less accurate predictions, resulting in a lower probability of predicting the next data point. For example, DeepLog [16] is a prediction-based system detecting abnormal logs in the file system. DeepLog converts each type of log entry into a unique discrete value referred to as a log key, followed by using a prediction-based LSTM to detect anomalies in the log key sequence.

2.2 Explainable Artificial Intelligence

Local and global explanations are two complementary approaches used in the field of Explainable Artificial Intelligence (XAI) to understand complex models. We first introduce the concepts and typical methods in general domains of AI and then introduce explanations of anomaly detection models and security-related domains.

Local and Global Explanation. Local explanation focuses on understanding the factors that contribute to a single prediction made by a model [50]. Typical local explanations can be further divided into three types. Approximation-based explanations leverage a simple linear model to locally approximate the complex model around the specific data point [13, 51]. Perturbation-based explanations find the important inputs by perturbing them with small noise and observing the major changes in the model outputs [11, 19, 20, 77]. Back propagation-based explanations are based on the gradient of the model outputs of certain inputs [55, 56, 59].

Global explanation, on the other hand, seeks to understand the overall behavior of a model [30]. Global explanations can also be divided into tree types. Global feature importance approaches quantify the overall impact of each feature on the model's predictions across the entire dataset [5, 21, 31]. Model decomposition approaches decompose the neural network to understand how different components of the model contribute to its global prediction behavior [5, 8]. Model distillation approaches simplify complex neural networks and replace them with more explainable models while maintaining predictive performance [29, 52, 65].

Explanation of Anomaly Detection. As mentioned in §1, the explanation of the anomaly detection model is essentially different from the supervised explanations. There are several local explanations for anomaly detection [4, 48, 57, 71]. However, most of them directly covert anomaly detection into supervised learning and leverage existing methods. As for others, COIN [40] provides a local explanation including important attribution and contextual reference for a group of similar anomalies. ATON [67] proposes a self-attention network that can identify important parts of anomalies. OC-DTD [33] extend [47] to locally decompose the one-class SVMs (OC-SVM). As mentioned in §1, local explanations are better at understanding specific anomalies but not the mechanism and knowledge of the entire model, thus not the scope of this study. As for global explanations of anomaly detection, Kopp et al. [36] use random forests to extract rules of anomalous samples, which needs supervised training. Barbado et al. [6] present a rule extraction technique over OC-SVM. However, the approach is hard to apply to deep learning models. KITree [38] is a recent global explanation method dedicated to anomaly detection models through distribution decomposition and boundary exploration, which leverages IID-based fidelity evaluation.

Explanation of Security Applications. Several studies have developed explanation methods dedicated to security applications [37, 63]. LEMNA [26] extends LIME [51] to explain anomalies detected by RNN-based models in security domains including binary reverse-engineering and PDF malware classifier. Cheng et al.[12] evaluate and improve local explanation methods for DL-based vulnerability detectors. DeepAID [28] proposes an explainer to identify the most influential features of anomalies for DL-based anomaly detection systems. x-NIDS [64] presents an approximation-based local explanation method dedicated to DL-based network intrusion detection systems (NIDS). CADE [70] and OWAD [27] propose explanations for finding the most important features and samples respectively when security classifiers and anomaly detectors encounter concept drift. However, the aforementioned methods are local explanations and most of them are designed for supervised learning models. As for global explanations, Trustee [32] proposes a tree-based explanation for DL-based supervised classifiers in the field of network security. Trustee uses partial training data to distill the original model as a decision tree and iteratively refine the tree with samples from the remaining training data with inconsistent decisions. However, as analyzed in §1, Trustee cannot be directly applied to anomaly detection and it only uses IID (training data) to fit and evaluate the tree model. Another related work is the Distiller

proposed in DeepAID [28], which combines expert feedback with the results of the explainer to form an FSM (finite state machine) based model. However, Distiller requires additional expert feedback, and it can only record part of abnormal rules instead of all the knowledge of the original model.

3 GEAD Explanation

In this section, we propose **GEAD**, a novel **G**lobal **E**xplanation method for DL-based **A**nomaly **D**etection models in security.

3.1 Overview and Motivation

The workflow of **GEAD** is shown in Fig. 2, which consists of six key steps. We briefly introduce the design and motivation of them. Firstly, in ❶, we treat the deep learning anomaly detection model to be explained as a black box, fitting a regression tree with the same training data as the original model as the input and the corresponding anomaly probabilities from the original model as the output. The derived regression tree is called *root* tree. Limited by the distribution of the original training data, the root tree is inconsistent with the original model well over samples in certain local feature spaces (or sampled from OOD distributions). Intuitively, some leaves in the root tree have inconsistent decisions with the original model. Therefore, in ❷, we locate such inconsistent leaves (or local feature spaces), which are called *low-confidence* (*LC*) leaves (or spaces). To eliminate the inconsistency caused by LC leaves, we need to apply fine-grained rules[2] for each LC leaf by continuing to *grow* the leaf with well-generated samples. In other words, we generate a *subtree* for each LC leaf. To do this, we need to generate an additional batch of training data for learning each subtree. The training set falling at the corresponding LC leaf is augmented by a well-designed generation strategy in ❸ and a subtree is fitted in ❹. Next in ❺, we replace each LC leaf in the root with the corresponding subtree, which derives a **merged** tree. All leaves in the merged tree contain probabilities that are similar to the original model. In practice, security operators focus more on the abnormal rules (or the rough degree of anomaly) rather than the specific value of anomaly probability. Therefore, we simplify the rules of the merged tree by binarizing all leaves into normal and abnormal based on the anomaly detection threshold of the original model. Leaves can also be split into multiple intervals to indicate different degrees of anomaly, which will be discussed later. Finally, in ❻, the simplified merged tree can be divided into normal and abnormal rule sets and reported to the operator.

3.2 Methodology

Below, we detail the six steps of **GEAD** and how to apply **GEAD** to two types of anomaly detection models (§2.1).

Regression Tree Generation. **GEAD** leverages the tree-based explainable model as it consists of *rules*. The form of rules is widely accepted in security domains and intuitive for security practitioners. Similar to previous works using tree models for global explanation, we utilize the same training set as the original anomaly detection model as the input of the tree model and use the output of the original model as the label to fit the tree model (called *root* tree).

[2]We interchangeably use the terms "*decision path*" and "*rule*" henceforth, and they both refer to a path from the root to the leaf in the rule tree.

Different from existing tree-based explanations for supervised models (like Trustee [32]) that directly utilize the binary classification outcomes (0 means normal and 1 means abnormal), **GEAD** trains a *regression* tree to approximate the output *probabilities* (i.e., anomaly scores) from the original anomaly detection models. This increases the fidelity by enabling the explainable model to capture the abnormal degree and distribution learned by the original model, not only the binary results. In this study, we leverage the well-known CART (Classification And Regression Trees) algorithm [41]. CART is an algorithm used for building trees for both classification and regression. When used as a regression tree, CART aims to predict a continuous value. The principle of a regression tree in CART involves splitting the input space into distinct regions, with the goal of minimizing the variance of the target variable within each region. CART regression trees use the mean squared error (MSE) as the default splitting criteria (also referred to as *impurity*) and then select the best split that minimizes the impurity after the split among all possible candidates iteratively. Since CART is not the contribution of this study, we refer readers to [41] for more details.

Low-confidence Leaves Identification. Recall the motivating example in Fig. 1, the root tree may have decisions inconsistent with the original model on the OOD samples. Therefore, **GEAD** locates low-confidence (LC) feature spaces, which refer to specific leaves in the root tree with a low level of decision-making confidence.

There are two types of LC spaces: one is the LC space of the original model, and the other is the LC space of the root tree. LC spaces of the original model can be measured by the anomaly probabilities. Take the reconstruction-based anomaly detection model as an example. The model is forced to reduce the reconstruction error of normal samples during the training phase. Thus, a lower anomaly probability value (reconstruction error) indicates that the sample is more widely present in the training distribution. Conversely, high anomaly probabilities are more likely to be OOD samples. So, we can directly select the regions (leaves) with higher anomaly probabilities as the LC space of the original model. LC spaces of the root tree refer to the not well-fitted leaves, which will directly lead to inconsistent decisions between the two models. And it can be simply measured by the *impurity* of each leaf. Specifically, we define two key parameters α_O and α_E to locate two types of LC spaces, representing the *quantile* of anomaly probabilities and impurities respectively. For example, if $\alpha_O = \alpha_E = 0.99$, then the 99th quantile of anomaly probabilities of all training samples and impurity of all leaves are set as two thresholds. Their selection will be discussed later. Consequently, root tree leaves with a decision value or impurity more than the corresponding threshold will be selected as the final LC leaves.

Low-confidence Augmentation and Tree Generation. After locating LC spaces, we eliminate the inconsistency within LC spaces by generating fine-grained rules for each of them. Intuitively, we continue to grow the LC leaves in the root tree with augmented data (original training data is not enough as the root tree growth has already stopped). To achieve this, we need to propose a well-designed data augmentation method based on the gradient of the original model. The high-level idea is to generate inconsistent samples under the guidance of the original model. Specifically, let $\mathbf{X}_i$ denote the samples in the training set that fall into the i-th LC

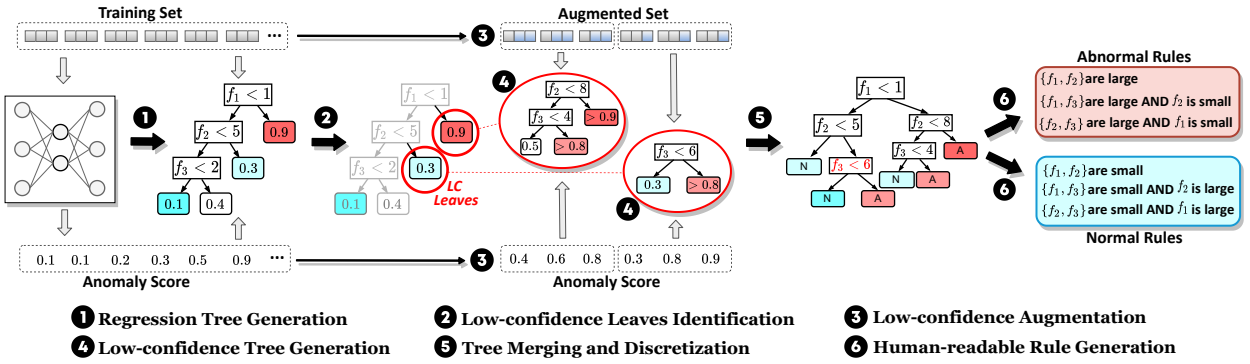


Figure 2: Overview of GEAD explanation.

leaves and $\mathcal{F}(\cdot)$ the original anomaly detection model. As the decisions from LC leaves are unreliable, we augment samples from two directions (normal or abnormal), denoted as $\mathbf{X}_i^-$ and $\mathbf{X}_i^+$, regardless of whether the original decisions of LC leaves. We use $\mathbf{X}_i$ in the original training set as the seed for multiple iterations of augmentation. $[\mathbf{X}_i^-]_j$ and $[\mathbf{X}_i^+]_j$ denote the set of normal or abnormal samples generated after j-th iteration. The gradient information of the model is used to modify $\mathbf{X}_i$ to find samples that lead to inconsistent decisions. Since the range of LC feature spaces is still large, gradient-based methods can augment the data more efficiently and with high fidelity to reflect the logic of the original model. Formally, for reconstruction-based anomaly detection, at the very beginning, $[\mathbf{X}_i^-]_0 = [\mathbf{X}_i^+]_0 = \mathbf{X}_i$. Then,

$$\begin{aligned}[\mathbf{X}_i^-]_j &= [\mathbf{X}_i^-]_{j-1} - \beta \cdot \frac{\partial \mathrm{MSE}([\mathbf{X}_i^-]_{j-1}, \mathcal{F}([\mathbf{X}_i^-]_{j-1}))}{\partial [\mathbf{X}_i^-]_{j-1}}, \\ [\mathbf{X}_i^+]_j &= [\mathbf{X}_i^+]_{j-1} + \beta \cdot \frac{\partial \mathrm{ReLU}\left(\sigma - \mathrm{MSE}([\mathbf{X}_i^+]_{j-1}, \mathcal{F}([\mathbf{X}_i^+]_{j-1})\right)}{\partial [\mathbf{X}_i^+]_{j-1}}.\end{aligned} \tag{1}$$

The second term in each line of Eq.(1) is computing the gradient of how to modify at each direction (negative or positive) and β is a hyperparameter indicating the stride of modification. In the negative direction, the gradient of reconstruction error to the samples at the previous iteration is computed and subtracted as we expect augmented samples with the small MSE. In the positive direction, besides flipping the direction of the (previous negative) gradient, we also need to limit the *upper bound* of MSE. It is difficult to decrease MSE but quite easy to increase it, which will lead to an *exploding loss* issue [15] if unconstrained. To solve this, we introduce the ReLU function and the upper bound σ (empirically set as 2 times the anomaly threshold of the original model according to [15]).

Note that in the tree-based models, the region corresponding to each leaf is a subspace of features. Since we only augment the samples in the subspace of LC, the range of augmented samples needs to be constrained. Therefore, when augmenting samples in a certain LC leaf, we retain the feature values that appear in the decision path leading to that LC leaf and only modify other features. Fig. 2 shows an example (❸ and ❹). For the first LC leaf (0.9 in red), since f_1 (the first feature) has already been used by the root tree, only the last two dimensions (f_2 and f_3) can be modified during augmentation. Similarly, the second LC leaf (0.3 in blue) can only modify f_3. This approach can keep the rules not in LC leaves unmodified. Therefore, we iteratively augment samples according to Eq.(1) and restore some features. The iteration will be conducted N times or until there are no non-zero gradients on the modified feature dimensions. Finally, for each LC leaf, the augmented samples from *all* iterations are used together to fit a regression tree (called *subtree*) in the same way as in the first step.

Tree Merging and Discretization. So far, we have obtained a root tree and some subtrees. Next, we need to merge them, and the process is quite straightforward as we ensure that the features in the decision path are not changed during augmentation. Therefore, we can directly replace LC leaves with the corresponding subtrees, resulting in a large *merged* tree. In practice, security operators focus more on the abnormal rules (or the rough degree of anomaly) rather than the specific value of anomaly probability. Therefore, we simplify the rules of the merged tree by discretizing the continuous value into ranges indicating different levels or degrees of anomaly. This process is also straightforward: we recursively (upwards from leaves to the root) determine whether the values of the two leaves belong to the same range, and, if so, merge them into one leaf. The division of ranges needs case-by-case analysis according to the specific security applications and requirements. For the sake of simplicity, in this study, we binarize the values into normal and abnormal with the detection threshold of the original model.

Human-readable Rule Generation. This is an optional step to increase the readability of the rules, enhancing the utility of **GEAD** in practical deployment. We can extract each rule from the discretized merged tree and sort the rule according to different abnormal degrees. The rule contains some triples (feature, decision threshold, and the relationship). For continuous features, security operators may not care about the specific decision threshold. Therefore, we can also discretize the continuous feature values within the rules. Fig. 2 (❻) shows a toy example where we split the continuous values into *small* and *large* to enhance the overall rule readability. Note that this step is optional and depends on the specific application.

Extending to Sequential Model. Tree-based models are intentionally used with *tabular* data. We have introduced the design of **GEAD** for tabular data and corresponding reconstruction-based models. Below, we will discuss the potential of **GEAD** for *sequential* or *time-series* data and prediction-based models. It should be noted that this is a preliminary first attempt and inevitably has some limitations. Future work can further investigate this. There are two significant differences between models trained with tabular and sequential data. The first is that the order of time points of

sequential data is immutable. Swapping the order of time points can lead to completely different decisions, while this is not the case for tabular data. The second is that the features in tabular data are generally independent of each other, but the time points of sequential data are interdependent. For example, if an important indicator is abnormal, the reconstruction-based model can make an anomaly decision relying purely (or mainly) on this one-dimensional feature. However, the predication-based model not only takes into account the current time point but also its *context*. Different contexts of a time point can lead to completely different decision results. The main challenge is to address the above two differences in rule-form (tree-based) explanations. To reduce the difficulty, we deal with univariate sequential models such as DeepLog [16]. **GEAD** is adapted through the following approaches. To address the first difference, we convert univariate time series into a tabular feature vector by its original order (e.g., a time series with a time window size of 5 is converted into a 5-dimensional feature). Each dimension represents a position of the time series, and therefore the position information is distinguished. To address the second difference, **GEAD** is enforced to involve all time points (feature dimensions) in rules, thereby preserving the contextual information. To achieve this, we additionally treat all leaves that are not fully grown as LC leaves. Another adaption of **GEAD** is Eq.(1) for data augmentation as we need to replace the MSE with the predictive values, but the high-level idea is the same. We provide more details in Appendix A.

Tree Pruning. Tree models employ pruning techniques to prevent overfitting on few samples and ensure generalization, especially when extracting rules, we need to ensure the *conciseness* of explanations. Some related works use post-pruning strategies (such as CCP in [44] and top-k in [32]). However, a significant issue is that the effectiveness of pruning is sensitive to the evaluation samples (which are still evaluated with IID in related work). Moreover, it lacks control over the total number of rules. In this paper, we opt for a simpler pre-pruning strategy, which predetermines conditions for stopping growth, including the maximum depth (M_d) and the minimum number of samples in leaves (M_s).

4 Evaluation

This section conducts the evaluation of explanations and provides several use cases and insights based on our explanation. First, we introduce the experimental setup including the datasets, anomaly detection models, and baseline explanations (§4.1). Then, we evaluate the quality of all explanations (§4.2). After proving the effectiveness of **GEAD**, we propose usage guidelines for **GEAD**, which tell what analysis can be performed on anomaly detection systems and showcase how to improve them (§4.3). Finally, we also provide several case studies and analysis of existing anomaly detection systems in security domains based on **GEAD** and conclude important insights for the community (§4.4).

4.1 Experimental Setup

Datasets and Anomaly Detection Models. We evaluate **GEAD** and baseline explanations upon two typical security applications, network intrusion detection and log anomaly detection, with four public datasets. As for network intrusion detection, we use the reconstruction-based anomaly detection model KitNET [45] and three widely-used public datasets: (1) Kitsune dataset [45] was collected by performing several attacks in an IoT network, and several statistics of traffic was used to extracted *packet-level* features; (2) CICIDS2017 [53] traffic contains benign behaviors and the several common attacks within a simulated enterprise network, and processes them into *flow-level* features; (3) Kyoto 2006+ Dataset [58] collects daily traffic from the honeypots deployed in Kyoto university network from 2006 to 2015, and processes them into *session-level* features. As for log anomaly detection, we use the prediction-based anomaly detection model DeepLog [16] and HDFS dataset [68] which collects the sequence of Hadoop file system logs. Two anomaly detection models are trained with the same settings as in their original work in the evaluation of §4.2. In §4.3, we change the hyperparameter and training method to improve them with the help of **GEAD**.

Baseline Explanation Methods. We evaluate the performance of **GEAD** with existing tree-based explanation methods and its components (i.e., ablation). We exclude local explanations and other forms of global explanations such as hyperplane [52] for their inconsistent purposes and representations. Regarding tree-based explanations, existing methods cannot be directly used in anomaly detection (e.g., Metis [44] for reinforcement learning models and Trustee [32] for supervised classification). Nevertheless, their core ideas are similar—distilling the DL model with a tree by selecting training samples and feeding the tree with outputs of DL models. Therefore, we extract the common part of existing tree-based methods as the baselines, including decision tree (DT) and regression tree (RT). We also consider different sampling strategies during distillation. For DT, abnormal samples are unavailable and not learned in anomaly detection models, thus, we randomly sample OOD anomalies as the training data. For RT, all training samples (IID) are used without LC identification, which can be regarded as *ablation* of **GEAD** to evaluate the effectiveness of LC augmentation and tree generation. We also evaluate the iterative sampling strategy from IID in Trustee [32] for both DT and RT[3]. In summary, we have four baselines:

- DT(IID+OOD): Train DT with the IID training set together with randomly sampled anomalies (labeled by the original model);
- DT(Trustee): Train DT with the IID training set while using Trustee's sampling strategy;
- RT(IID): Train RT with the IID training set, which is an ablation of **GEAD** without LC augmentation;
- RT(Trustee): Train RT with a pure normal IID training set while using Trustee's sampling strategy.

Besides, KITree [38] is another baseline as the representative of explanations dedicated to anomaly detection[3].

Metrics. The common metrics for evaluating the quality of explanations are *fidelity* and *conciseness*. Fidelity means the similarity (of decisions) between two models. A common practice in existing work is using *accuracy* to evaluate the similarity between the explainable and original models. However, we argue this metric is insufficient to fully observe the explainable model's fidelity and distinguish between normal and abnormal decisions. Therefore, we propose four fine-grained metrics:

[3]Note that, the results of Trustee-related baselines and KITree in this study **do not necessarily** hinder or contradict their effects in the original papers [32, 38] as we already modified Trustee for anomaly detection and use OOD instead of IID for evaluation.

Table 1: Fidelity evaluation (consistency and credibility) of explanation methods (*higher is better*).

Explanations	(a) CICIDS2017				(b) Kitsune				(c) Kyoto2006+				(d) HDFS			
	Consistency		Credibility		Consistency		Credibility		Consistency		Credibility		Consistency		Credibility	
	Positive	Negative	Positive	Negative	Positive	Negative	Positive	Negative	Positive	Negative	Positive	Negative	Positive	Negative	Positive	Negative
DT(Trustee)	0.494	0.997	0.966	0.920	0.298	0.996	0.998	0.124	0.146	0.987	0.933	0.495	0.881	0.984	0.813	0.990
DT(IID+OOD)	0.509	0.996	0.966	0.922	0.741	0.997	0.999	0.277	0.146	0.987	0.933	0.495	0.873	0.988	0.849	0.990
RT(Trustee)	0.603	0.992	0.931	0.936	0.809	0.994	0.999	0.342	0.915	0.991	0.992	0.908	0.790	0.997	0.954	0.984
RT(IID)	0.882	0.977	0.871	0.980	0.808	0.997	0.999	0.341	0.916	0.991	0.992	0.909	0.871	0.997	0.958	0.990
KITree	0.447	0.998	0.986	0.914	0.990	0.985	0.999	0.901	0.686	0.342	0.551	0.481	0.908	0.983	0.812	0.992
RT(**GEAD**)	**0.923**	**0.998**	**0.987**	**0.987**	**0.994**	0.997	0.999	**0.946**	**0.982**	**0.989**	**0.990**	**0.979**	**0.911**	**0.998**	**0.976**	**0.993**

- *Consistency*: From the perspective of the original model, how many of the same *positive* or *negative* predictions does the explainable model have?
- *Credibility*: From the perspective of the explainable model, how many of the *positive* or *negative* predictions are inconsistent?

For example, *Positive Consistency* is the proportion of the same positive decisions between the two models to all the positive decisions of the *original* model, and *Negative Credibility* is the proportion of the same negative decisions between the two models to all the negative decisions of the *explainable* models. As for *conciseness*, the *number of rules (branches)* in the tree is used in [32]. There are also other metrics to evaluate the explanations in related research [18, 28, 63] including stability, efficiency, and robustness. We leave the result and discussion of them in §6.

Hyperparameters. There are six hyperparameters for **GEAD**: LC thresholds α_O, α_E, stride β, number of iterations N, and M_d, M_s for pre-pruning Note that, the value of N is directly related to the runtime, and we show the insensitivity of N and β within a certain range in Appendix B.1. We perform a grid search on $\alpha_O, \alpha_E, M_d, M_s$ to meet the requirements of conciseness. The same search of M_d, M_s is used for trees in baselines.

4.2 Quality of Explanations

Evaluation Methods. There is a trade-off between the fidelity and conciseness of the explanation. The state explosion can occur during tree generation without constraints on the tree size, resulting in the generation of thousands of nodes and, consequently, poor readability of rules. Therefore, we aim for an explanation to achieve better fidelity with fewer rules. To this end, we first set a reasonable budget for the number of rules, i.e., the maximum number of rules the security operator is willing to investigate, which is determined as needed in practice. Additionally, we evaluate how fidelity varies with the number of rules.

Results. We first evaluate the fidelity of all explanation methods within a limited budget of conciseness. For tabular cases (i.e., the first three datasets), the maximum rule number is 100. For the sequential case (i.e., HDFS), the maximum rule number is 300 (As mentioned in 3.2, explaining sequential model generally needs more rules). As mentioned in the §1, IID samples cannot reflect the real fidelity. Thus, we evaluate consistency and credibility with a combination of IID and OOD samples. For example, in the Kyoto dataset, KitNET is trained with the normal traffic traces in 2007 and then evaluated with traces selected from 2007 to 2015. The results of fidelity (including consistency, credibility, and the decrease of ROC AUC after feature flipping) under four datasets are listed in Table 1. The results show the effectiveness of **GEAD** over baselines.

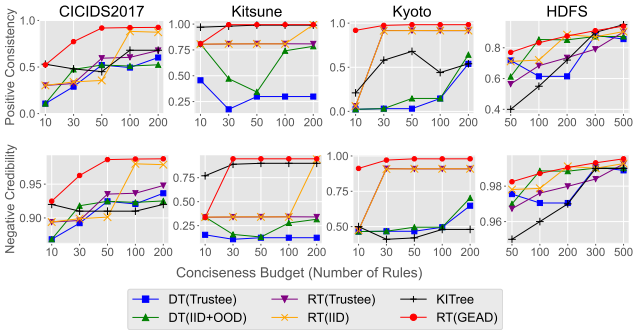


Figure 3: Evaluate the quality of explanations under different conciseness budgets in four datasets.

On one hand, RT-based methods are better than DT-based ones, which proves that the anomaly probabilities can improve fidelity. On the other hand, **GEAD** with LC sampling is better than RT and IID-based (Trustee) methods, which proves the effectiveness of OOD-based LC sampling. Based on the results, we also evaluate the trade-off between fidelity and conciseness by observing the most differential fidelity metrics (i.e., positive consistency and negative credibility) for all datasets as the conciseness budget changes. As shown in Fig. 3, we can see that **GEAD** can achieve better fidelity with fewer rules compared with baselines. An exception is that in the Kitsune dataset, KITree outperforms **GEAD** with 10 rules, but is surpassed by **GEAD** when the number of rules increases.

4.3 Usage Guidelines for GEAD

After proving the effectiveness of **GEAD**, we showcase how to improve existing DL-based anomaly detection models based on our explanation. Fig. 4 shows a usage guideline for **GEAD** upon anomaly detection. We propose three kinds of usage:

(1) **Model vs Expert**: We compare the rules extracted from anomaly detection models with common sense and expert knowledge based on the background of security applications.
(2) **Model vs Model**: We compare the knowledge of anomaly detection models under different settings including different training data, model hyperparameters, and training tricks.
(3) **Model + Expert**: We combine the rules from the expert system and the advanced knowledge learned from DL models to build a more reliable rule set for anomaly detection.

The first two kinds of usage (Model vs Expert/Model) can help to build more reliable anomaly detection models for security applications, while the last usage can combine the advantages of rule-based

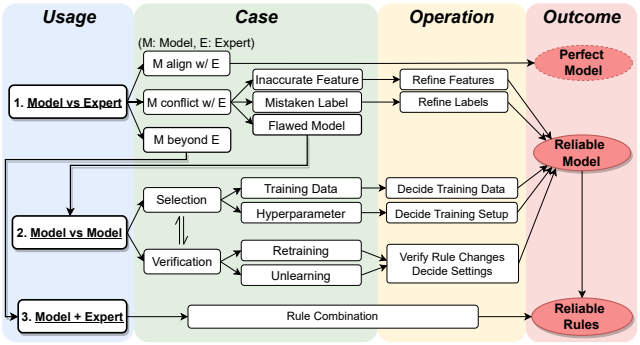


Figure 4: Usage guideline based on GEAD.

and anomaly-based detection. For each kind of usage, we further list possible situations (see "***Case***" in Fig. 4) and give corresponding operation suggestions ("***Operation***" in Fig. 4). Below, we will showcase each kind of usage of **GEAD**.

4.3.1 Usage #1: Model vs Expert. Here, we analyze the *correctness* of knowledge learned by anomaly detection models by comparing the rules of **GEAD** with common sense. One may be concerned about the authenticity and fairness of Expert knowledge. To eliminate this, we do not actually ask experts to generate some rules and then compare them with model rules. Instead, we observe the correctness of the model rules extracted by **GEAD** to *detect specific attacks or anomalies* based on the background of the corresponding dataset and security applications. Such analysis inevitably requires manual effort, and future work can look for more efficient or automated methods.

When a model's knowledge (rules) completely aligns with expert expectations, we obtain a perfect model. However, experiments demonstrate that such a situation is almost impossible in practice. When a model's rules do not align with expert expectations, two further cases arise: the first is the model learns valid rules beyond expert expectations, which is a promising outcome and sets the stage for the third usage (i.e., Model+Expert). The second is that the model learns incorrect rules. In this study, we identify three potential reasons: *inaccurate features*, *mistaken labels*, and *flawed models*. We will further discuss them below.

Inaccurate Feature. This happens because the features contained in the dataset are not extracted accurately as expected. For example, in the CICIDS2017 dataset, we train KitNET with normal traffic on Tuesday to detect DDoS traffic on Friday. The ROC AUC of the model is 0.75, which is extremely poor (aligned with the results in [42, 72]). Here, we would like to investigate the reason through **GEAD**. Note that, the extracted rules consist of various types of anomalies and we only focus on the rules for detecting DDoS attacks in this case. There are two main branches of DDoS attack traffic samples: one is TP ($\approx$ 25.3%) and the other is FN (i.e., a benign model rule but actually wrong). We compare the consistency of decisions on these two types of samples between the original model and **GEAD** tree, and the result is 100%, thus demonstrating the fidelity. We list two types of rules in the red box below, with the "context" representing the common aspects of both rules:

DDoS Rule (KitNET on CICIDS17)

(1) `Init_Win_Bytes_Fwd` $\neq -1 \cap \in (0, 50000)$
(2) `# SYN, URG, FIN` $= 0$
} **Context (Common)**

(3) `Init_Win_Bytes_Bwd` $\in (2, 40000)$
(4) `Pkt_Len_Var` $> 1E7$
(5) `Active_std` < 40000
} **True Positive**

(3) `Init_Win_Bytes_Bwd` $= -1$
(4) `Pkt_Len_Var` $< 8E6$
(5) `Bwd_IAT_Std` $< 4E7$
} **False Negative**

The whole tree is shown in Appendix C, where we also list the description of related features in the rules. The features from top to bottom in the box are in the order from root to leaf nodes. Note that, although we only list DDoS rules here, they can still reflect the overall decision-making philosophy of the model (e.g., features closer to the root are more important for distinguishing samples with different anomaly scores).

After investigating the DDoS traffic generation and feature extraction in [54], we find at least two mistakes in feature extraction: First, whether `Init_Win_Bytes_Fwd` or `Init_Win_Bytes_Bwd` (the total number of bytes sent in the initial window in the forward or backward direction) is -1 should not be the key point to detect the DDoS as -1 is the missing value of this feature during extraction. However, most DDoS traffic has a value of -1, leading to false negative decisions; the second one is that `#SYN, #URG, #FIN` (the count of SYN/URG/FIN flags within a TCP flow) = 0 is counter-intuitive to be the second most important feature because it's improbable for a large number of flows to lack SYN or FIN packets. Upon investigation, we find that all features related to TCP flags are not correctly extracted in the original CICIDS2017 dataset and only the first packet in each flow is considered. These two mistakes have been validated in recent works [17, 39]. We leverage their *improved* CICIDS2017 dataset to correct the feature extraction and labels (discussed later). After that, we retrain the autoencoder with the improved features under the same setting as before. The ROC AUC increases from 0.75 to 0.90, and the recall of detecting DDoS improves from 25.3% to over 99.9%. We also list the new detection rules below (the whole rule tree is in Appendix C):

DDoS Rule (KitNET on Improved CICIDS17)

(1) `Bwd_Pkt_Len_Min` ≤ 3.5
(2) `Bwd_Head_Len` > 10
(3) `Pkt_Len_Var` $> 1E6$
(4) `Init_Win_Bytes_Fwd` < 22000
} **Context (Common)**

(5) `Pkt_Len_Std` > 2300 ← **Rule 1 (True Positive)**
(5) `Bwd_Seg_Size_Avg` > 1700 ← **Rule 2 (True Positive)**

There are also two rules to detect DDoS (different from before, both are TP). We can find that the model rules trained by the improved features are more reasonable. By analyzing the raw traffic, we found that normal and abnormal traffic indeed differ in relevant features, especially volume-related features in backward traffic. The possible reason is that regular network traffic typically includes more small-sized stable backward segments, while under LOIC DDoS (the tool used in the dataset), servers attempt to send larger backward segments comprising error messages and timeout responses, thus responding differently from normal cases. Fig. 5 shows the distribution of representative features in the rules, and we can find that the features in the two plots on the left can well distinguish normal and DDoS traffic. There are also features that seem to have little semantic connection with attack behaviors, like `Init_Win_Bytes_Fwd`. However, as shown on the right of Fig. 5, an

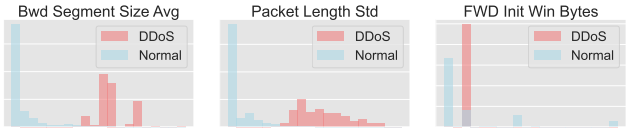


Figure 5: Distribution of features in DDoS rule.

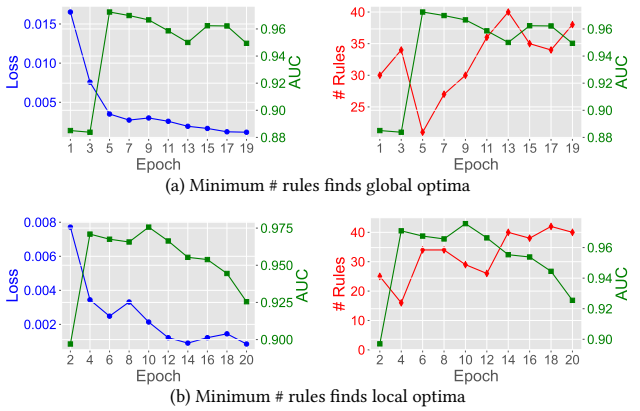
interesting finding is that the initial window of all DDoS traffic is 8,192, which is the default setting of the attack tool. Note that, we do not think this is a *shortcut* learning, because the anomaly detection model has not seen DDoS traffic during training (has no idea about the 8,192 window size). Instead, we tend to believe that the model learns that `Init_Win_Bytes` is able to *fingerprint* anomalies by observing the normal traffic.

Mistaken Label. This happens when samples in the dataset are not labeled correctly. For example, when detecting Mirai attacks in Kistune, we found that many attacks were not detected based on the labels in the original data set. After interpretation with **GEAD**, we find that these "false negatives" are quite similar to the benign data. We highlight the rules/decisions of the model tested with normal and "Mirai" samples respectively in Appendix C. The decision paths are quite similar for the two kinds of traffic. Motivated by the similar decision paths in **GEAD** rule tree, we analyze the original traffic and find that all packets after launching the attack process are labeled as malicious (accounting for 84% of the entire data set). However, a lot of benign background traffic is also mixed during this period and mislabeled. Therefore, we relabel the attack traffic based on the attacker's IP and attack behavior and retrain the anomaly detection model with the correct labels and the same hyperparameters. The performance of the new model is significantly improved (especially for reducing FN, TPR is increased from 0.88 to 0.95). The whole rule tree with highlight decisions is shown in Appendix C. We can find that most of the "false negative" decisions are eliminated.

Flawed Model. This happens when the anomaly detection model is not well-trained, such as under-fitting, over-fitting, or out-of-date due to distributional drift. To further investigate the impact of different learning methods on the knowledge learned by the model, we introduce the second usage of **GEAD**: Model vs Model, which is to compare the rules between models through different learning.

4.3.2 Usage #2: Model vs Model. Another usage is to compare the difference of rules for different models to gain insights into model construction. Here, we present two types of usage: *selection* and *verification*. The first is to extract the model knowledge of different training data to answer how to select model hyperparameters, and *which* and *how much* normal training data to select. The second is to verify two commonly used techniques for continuous learning of anomaly detection: *retraining* [3, 27] and *unlearning* [15, 74].

Hyperparameter Selection. Hyperparameter selection is one of the key factors affecting the performance of DL models. Here, we focus on the training epoch (i.e., when to stop training) as it is a special case for anomaly detection. There is no such thing as a validation set containing abnormal samples in the zero-positive setting. Thus, one can only determine when to stop training through experience or training loss. The selection of other hyperparameters (e.g., learning rate, optimizer) is omitted here as it is widely discussed in the DL community. Here, we use the improved CICIDS17 dataset



(a) Minimum # rules finds global optima

(b) Minimum # rules finds local optima

Figure 6: The relationship between training loss, AUC, and the number of rules during training (KitNET on Improved CICIDS17).

Table 2: Comparison of methods for deciding when to stop training.

Method	$\mathbb{P}(local)$	$\mathbb{P}(global)^\star$
Early Stop (when Δloss<1‰)	11%	1%
Randomly Select (after Δloss<1‰)	26%	3%
Last Epoch (minimum value of loss)	27%	3%
Ours (minimum number of rules)	75%	59%

⋆ Here "global" means the global optimal *within all training epochs*

for the case study. We first investigate the relationship between loss and AUC during training, as shown in the left two plots of Fig. 6, their relationship is not linear: the model performance may fluctuate. as loss decreases. This means that we cannot decide when to stop training just by looking at the loss. We try to find better solutions by observing the changes in the internal rules of the model (via **GEAD**) in different epochs. Their rule trees are listed in Appendix C. An interesting finding is that *the number of rules contained in a model tends to be inversely proportional to its performance*, as shown in the right two plots of Fig. 6. After analyzing different rule sets extracted by **GEAD**, we find that the shape of the rule tree and the number of rules can reflect how well the model fits the training data. At the beginning, the number of rules decreases and important rules are retained to overcome the under-fitting; as the training epoch increases, the model may become over-fitting, and generate more leaves in the rule tree.

To verify the generalizability of this finding, we experiment on deciding the training epoch. The proposed method based on the above findings is to select the model with the minimum number of rules. We also introduce some baselines. The first one is to directly select the last training epoch. The second one is to stop early (a common practice in the DL community). That is, stop training when the reduction of loss is less than a threshold (one-thousandth of the initial loss in this study). The third is to randomly select an epoch to stop. The selection starts from the early stop epoch to avoid selecting very initial epochs. We randomly select hyperparameters (except epoch=20) and repeat the experiment 100 times on the Kistune dataset to evaluate the proportion of the epochs obtained by different methods that are the same as the "standard answer". The

standard answer includes global optimal epoch and local optimal epoch(s). For example, in the right of Fig. 6(a), the global optimal epoch is 5, and local optimal epochs are 5 and 15. The results are listed in Table 2. We can find that using our simple method can significantly improve the probability of finding the optimal or sub-optimal training models among all epochs.

Training Data Selection. When training an anomaly detection model, we may have a large number of candidate normal samples on hand, but it is often confusing *how many* and *which* samples to select as the training set. Therefore, we compare the knowledge of models trained with different normal data. We first answer the first question (*how many training samples*). Here, we use the Kistune dataset for the case study. One might have the intuition that more training data is better. However, as shown in the right/green line of Fig. 7, the relationship between training size (x-axis) and the model performance (AUC) is not simply linear, which brings difficulties to the selection of the number of training sets. To investigate this, we observe models trained with different training sizes by extracting the rules through **GEAD**. Their rule trees are listed in Appendix C. Similar to the experiment of hyperparameter, the number of rules is again inversely proportional to model performance, as shown in the left/red line of Fig. 7. After analyzing different rule sets extracted by **GEAD**, we find that it is possible to well fit the training distribution on a very small training set (see the rules under 5k), and the performance is not poor when the distribution is simple and not drifting. As the number of training sets increases, the model tends to perform better. However, too much benign data (see the rules under 200k) is likely to cause the model to *overfit* to the benign distribution, thereby thinking that everything is normal, thus showing an under-fitting phenomenon in turn. Motivated by the previous experiments, we also select the number of training sizes by referring to the number of rules. Similar to the previous one, we repeat the experiments in Fig. 7 100 times with different hyperparameters. The result is that our method hits the optimal training size 72 times (72%), while selecting 5K and 20k (minimum or maximum) is both lower than 50%.

Regarding the second question (*which samples to select*), the intuition is to ensure that the training data covers all the benign sample space. However, it is difficult and even impossible to cover quite a large space at once. Therefore, we consider leveraging the concept of low-confidence (LC) space in **GEAD**. The difference is that we augment the original training set with data falling in the LC leaves to improve the performance of the original model (instead of explaining it). Specifically, we randomly select a part of the training data to train the model, call **GEAD** to locate the LC leaves, and augment the original training set with remaining samples falling on the LC leaves. Such a process is repeated T times. To evaluate the effectiveness, we use randomly selected training sets as a naive baseline. Another baseline is trying to cover benign space at once based on the idea of stratified sampling. To simplify the complexity, we use the one-dimensional distribution of the model outputs instead of the original feature distribution (motivated by [27]). We divide the value range evenly (100 parts in this experiment) and ensure that the number of samples in each interval is similar. The experimental results are shown in Table 3. It can be seen that our method has the best performance, and increasing the number of repeated times can further improve it.

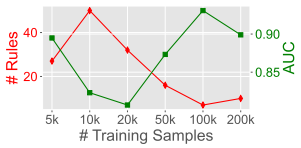


Figure 7: AUC and the number of rules under different training sizes.

Table 3: AUC under training dataset selection methods.

Method	AUC
Randomly Sampling	0.89
Stratified Sampling	0.92
Ours (LC Sampling $T = 1$)	0.95
Ours (LC Sampling $T = 10$)	**0.96**

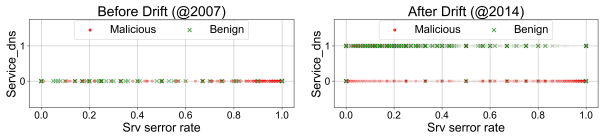


Figure 8: Distributions of two features before and after drift.

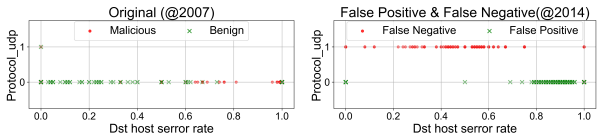


Figure 9: Distributions of two features used in UNLEARN.

Verification of Retraining. DL models are susceptible to concept drift—the data distribution changes over time and moves away from the original training distribution. Particularly, Anomaly detection is affected by changes in the distribution of normal data [27]. To cope with this, the common practice is to regularly update the model to ensure it has learned up-to-date distribution. In this case, we verify whether the model learns the expected knowledge by comparing the rule changes before and after the model retraining. Two representative methods are introduced here. The first is to directly retrain the old model with data from the new distributions (noted as *Retraining*), and the second is the state-of-the-art concept drifting handling method called OWAD[27]. We use the Kyoto dataset as it consists of a long period. The model is trained with normal traffic in 2007 and tested with traffic in 2014. As verified in [14], there is a significant drift from 2007 to 2014, and a significant degradation in model performance is witnessed. The AUC is 0.96 in 2007 but only 0.29 in 2014 (ROCs are in Fig. 10 of Appendix B).

We retrain the original model with 2014 data and update the model with the default settings of OWAD. The AUC on the test set of 2014 after Retraining is 0.60 and 0.83 after OWAD. We list the original model (trained with 2007 data) as well as the updated models by the two methods in Appendix C, where we also highlight the decision paths for the shifted anomalies in 2014 traffic. After investigating these rules, We find that OWAD identifies two most important features that cause drift, `Srv_serror_rate` and `Service_dns`, and the latter is considered more important (how to determine the importance of features in GEAD rules will be introduced in §4.4); However, Retraining only finds `Srv_serror_rate`. We analyze the raw traffic and confirm that there is almost no DNS traffic in the normal traffic in 2007, while the majority of the normal traffic in 2014 is DNS traffic, verifying that `Service_dns` is an important feature that causes drift. Fig. 8 plots the distribution of the two

features before and after drift. It can be seen that `Srv_serror_rate` after drift is insufficient to distinguish between normal and abnormal data while `Service_dns` is sufficient. As analyzed in [27], the disadvantage of retraining is that it cannot ignore the out-of-date knowledge in the old distribution. In summary, we showcase here how to verify the effectiveness of OWAD with the help of **GEAD**.

Verification of Unlearning. Another technique for anomaly detection models to be continuously updated over long-term deployment is UNLEARN[15]. It assumes that the operator has found some FP/FNs and would like to update the model with them. It is supposed to only update the parameters relevant to the wrong decisions without affecting the other correct knowledge (known as *catastrophic forgetting*). Specifically, UNLEARN solves an optimization problem to find a trade-off between two contradictory terms: updating the model to correct FPs or FNs (the learning term) while limiting the update of important model knowledge (the forgetting term), where the forgetting term is balanced by a parameter λ. The smaller λ is, the less restricted the update of model parameters is.

In this case, we leverage **GEAD** to observe how the rules of the model change before and after UNLEARN and how the knowledge changes for different λ. We still use the model trained on Kyoto 2007 data from the previous case and test it in 2014. We then randomly select 100 FPs and FNs to UNLEARN ($\lambda = 1$ here). The original and updated models using FPs and FNs are plotted in Appendix C. We find that about half of the rules (the right part of the rule tree) are unchanged (not only features but also thresholds on the decision path), which indicates that the forgetting term in UNLEARN indeed limits catastrophic forgetting and validates the feasibility of updating part of the model knowledge. The two most important features that are updated in the model rules are `Protocol_udp` and `Dst_host_serror_rate`. As in the previous case, we plot the distributions of the two features (shown in Fig. 9). We can observe that these two features have changed significantly in 2014 and need to be updated, which also verifies the correctness of the learning term of UNLEARN.

Next, we investigate the impact of λ in UNLEARN. Four different values are selected: $\lambda \in \{10^{-6}, 10^{-3}, 1, 10^{3}\}$. The rules of different models are shown in Appendix C. We evaluate the knowledge forgetting by testing models of different λ with TNs of the original model. The model forgets important knowledge if original TNs are mislabeled as anomalies. The results and extracted rules show that when λ is too small ($10^{-6}, 10^{-3}$), the model indeed forgets a lot of correct knowledge. An interesting observation is that the model works just as well even if picking a large λ (10^{3}), which means restricting the forgetting term is more important than the other. In summary, we showcase that **GEAD** can intuitively observe the changes in model knowledge, which helps security operators to establish trust in model decisions.

4.3.3 Usage #3: Model + Expert. Integrating rule and anomaly based methods is a straightforward idea and has been well-studied [22, 25, 34, 46, 75]. However, prior work mainly integrates the prediction results of the two methods. By contrast, the advantage of our study is that **GEAD** can extract the knowledge contained in the DL-based anomaly detection models to achieve an explicit and deep fusion of rules. We showcase in §5 with a real-world application.

4.4 Summary of insights

We summarize key insights of DL-based anomaly detection in security based on the previous analysis and case studies with **GEAD**.

> **Insight 1**: *Decision Tree-based explanations cannot correctly reflect the decision-making logic in original DL-based anomaly detection models, while Regression Tree-based explanations with only IID samples tend to ignore or misrepresent part of the original logic.*

Claim of Insight 1. This insight has been validated by experiments in §4.2, and we also provide a comparison of rules between **GEAD** and baselines in Appendix C. This insight holds significant importance because if the original behaviors are not accurately extracted, the conclusions drawn from the rules cannot reflect the original model. Instead, they would represent a new model trained on the same dataset but with altered behaviors.

> **Insight 2**: *Different from the prevalent phenomena of **shortcut learning** in supervised learning (reported in [32]), DL-based anomaly detection rarely predicts with just a few features.*

Claim of Insight 2. In [32], Jacobs et al. find that there is a common phenomenon of *shortcut learning* in *supervised* models in the network security domain, that is, the decision logic of the supervised model can be represented by a few rules (e.g., achieving over 99% fidelity with only 10 rules[4] This is not a good phenomenon as the model will predict through shortcuts (e.g., defects in the data collection) rather than expected knowledge. However, shortcut learning is *not* prevalent in anomaly detection models. As shown in Fig. 3 of §4.2, it is impossible to describe the original decision logic with only a few rules (e.g., 50 rules in Kyoto and 100 rules in CICIDS17). We believe this is due to the regression-based training mechanism of anomaly detection (as mentioned in Insight 1). The good news is that this shows that the introduction of DL does make the anomaly detection model perform more complex reasoning, while see the next Insight 3 for the bad news.

> **Insight 3**: *A prevalent phenomenon is observed wherein rules in anomaly detection models are in conflict or unexpected with established common sense. Therefore, regarding **outliers** from benign data as genuinely **malicious** remains premature.*

Claim of Insight 3. In most of the experiments in this study, we can find that the rules in anomaly detection models are not exactly as expected. One reason is the mistaken labels/features mentioned in §4.3.1 or a not well-trained model mentioned in §4.3.2. But even if we fix the above issues, there are still unsatisfied rules. For example, in the case of Fig. 9, the model distinguishes malicious traffic mainly through whether it is UDP traffic as there is almost no UDP traffic in benign traffic in Kyoto 2014. Another example is the detection of LOIC DDoS attack in CICIDS17 in §4.3.1 mainly uses the initial time window size wherein the attack software is fixed. As anomaly detection is learned and detected entirely from the normal data without prior knowledge about the attack, any samples that deviate from the training distribution will be judged as anomalies. This is also why there is no shortcut learning in Insight 2 as the

[4]We evaluate fidelity of DT-based explanation also with OOD instead of IID [32], this is why the DT methods do not exhibit shortcut learning in Fig. 3.

distribution of anomalies can be quite broad. Nevertheless, security operators can obtain more insights for data distribution from the model knowledge, while they need to be careful when deciding whether an anomaly is a real threat or not.

Insight 4: *Regarding the importance of rules (or features) in the decision path, for the entire model, the rules closer to the root node are more important; however, for a specific decision, the rules closer to the leaf node are conversely more important.*

Claim of Insight 4. This insight provides an approach to finding important features within certain rules. The first part (i.e., for the entire model) is well demonstrated in CART [41]. To help operators analyze specific decision rules (e.g., in Fig. 8 and Fig. 9), we propose the second part (i.e., for a specific decision) of insight. From the mechanism of CART, the immediate predecessor feature for a specific leaf node is significant as it directly determines the partitioning that leads to that leaf node. An intuitive example is Fig. 5, where the feature closer to the leaf has more distinguishable normal and abnormal (LOIC DDoS) distribution. We also empirically justify this through two experiments in Appendix B.3.

Insight 5: *Comparing anomaly detection models trained under different settings, the number of rules contained in the model is often inversely proportional to the detection performance of the model.*

Claim of Insight 5. This insight is justified empirically in Table 2 of §4.3.2, which is an easy way to select a well-trained model. Note this approach is not completely correct and cannot be theoretically guaranteed (as fully understanding the relationship between training and knowledge change is still an open problem). Future work could investigate this and propose more reasonable approaches. In applications with low error tolerance, security operators are encouraged to perform a case-by-case analysis based on the rules.

5 Real-World Use Case

To demonstrate the potential of **GEAD** in real-world anomaly detection systems, we have worked with a large company responsible for supplying electricity transmission and unified control of the power grid in China to showcase how **GEAD** can help to improve its security monitoring system.

Expert Knowledge. The supervisory control and data acquisition system (SCADA) will continuously monitor all devices and generate logs in the following format:

<level> <timestamp> <device> <event category> <event info>,

which can generally be divided into two categories: The first category of logs regularly records device-related information (such as CPU utilization and fan speed). The second category of logs is triggered by events that happened in certain devices (such as USB (un)plugging and directory changes). Each log will be classified into *dangerous*, *important*, and *common* according to its type and specific information, as the *<level>* field. Dangerous and some important logs will be reported to the operator. Exactly, the classification of logs via level is a rule set based on expert knowledge.

Model Knowledge. In addition, deep learning-based anomaly detection systems have also been built to detect anomalies in log sequences. The detection model here is still DeepLog [16]. We consider both log type and information when processing the sequence of log keys. Continuous *<event info>* fields like CPU rate are binned as 10 discrete keys. We use **GEAD** to extract the sequential rules in the trained DeepLog, thus obtaining the rule set of the model.

Table 4: Combining model and expert knowledge.

	Device 1		Device 2	
	FP	FN	FP	FN
Expert Rule (*dangerous*)	**0**	1,872	**0**	492
Expert Rule (*dangerous*, *important*)	19,786	821	161	425
Model Rule (extracted by **GEAD**)	1,430	259	42	93
Model + Expert	94	**217**	**0**	**64**

Data Collection and Labeling. We collected over 1.3 million logs from two typical devices (a gateway and a workstation) from October 2021 to March 2022 and during June 2023, and let the operators help label the data. Note that, we need to define what exactly is an "anomaly" or the expected anomaly in the context of the grid. First of all, *dangerous* level logs must be marked First of all, dangerous level logs must be marked as abnormal (accounting for ∼0.03%) as they contain certain dangerous events such as virus logs. However, *important* level logs cannot be directly marked as anomalies (accounting for ∼7.9%) as they contain a large number of uninterested events, and it is labor-intensive to check all of them. Indeed, more than 99% of the logs with the important level are uninterested events after being checked by operators. For example, operators are not concerned about a CPU utilization increase due to a surge in the number of tasks but will be concerned about an increase due to device failure. This is also the reason why relying solely on the *level* field is not accurate enough, leaving the opportunity to combine model and expert knowledge.

Rule Combination. Therefore, we combine the ability of the *model rule* to learn log sequence context with the domain-specific prior knowledge of the *expert rule*. Specifically, we use **GEAD** to extract the rules of trained DeepLog and mainly focus on abnormal rules (the same as the expert rule). The combination of model and expert rules can be divided into three cases. The first is that the model rule includes *dangerous* level log, which is consistent with the expert rule, so all of them are retained and simplified into single-feature rules (∼7% of all model rules). The second is that the model rule includes *important* level logs. The principle is to filter uninterested anomalies based on the context (∼36% and ∼52% are retained). For example, high CPU utilization will be an anomaly during routine device information reporting, but will not be after a large number of legitimate transmission services are started. The third type is model rules that do not contain dangerous or important logs. Such rules need to be judged case by case whether they are anomalies of interest (∼57% and ∼14% are retained). For example, the order of certain important device services deviates greatly from the normal situation. Due to data compliance and privacy restrictions, we cannot release the original rules (whether expert, model, or combined) to avoid exposing commercial secrets or equipment status. We list an example desensitized rule tree in Appendix C).

Results. We use 50% logs to train DeepLog, 20% to verify the model and establish **GEAD**, another 20% to verify **GEAD** and the rule combination, and the remaining 10% is to evaluate the rule combination,

as shown in Table 4. We can find that although checking dangerous level logs will not cause false positives, it will miss a large number of important anomalies (e.g., Device 2 does not have any logs with a dangerous level). Meanwhile, introducing the important level will also introduce a large number of false positives. Model rules can effectively balance the false positives and false negatives, while rules combining model and expert knowledge achieve the best trade-off.

6 Discussion

We discuss the other aspects of performance and design considerations of **GEAD**, as well as future work.

Stability, Efficiency, and Adversarial Robustness. The stability of the Trustee must be ensured by growing multiple (usually 50) trees and measuring their agreement. By contrast, **GEAD** eliminates random factors to ensure the stability of rules. We deem that stability is also a part of fidelity, once the original model is trained, its rules should be deterministic instead of affected by the explanation. The runtime of explaining the CICIDS17 dataset is 3s, 2407s, and 3170s for **GEAD**, Trustee, and KITree with default hyperparameters[5] on Ubuntu 16.04 LTS with 16 vCPUs. This is because we do not need to repeat sampling to ensure stability. We believe that adversarial robustness is not a potential issue for **GEAD** because the rule extraction is conducted by the operator offline and the attacker cannot access it unless hijacking **GEAD**. The attacker needs full control of the anomaly detection system or the rule extraction procedure in **GEAD** to perturb the final results of the rules. However, this assumption is extremely hard and not the case for discussing robustness in related works [20, 23, 28] as we must ensure the algorithm itself is not modified. Even if the DL model is poisoned [49], **GEAD** can still accurately extract its poisoned behaviors.

Design of GEAD. Firstly, **GEAD** focuses on fidelity instead of accuracy as we would like to extract the knowledge of the anomaly detection model, regardless of its performance. This is different from existing supervised explanations [32, 44] that increase the accuracy, as the tree model itself is supervised and cannot apply to zero-positive anomaly detection. Secondly, one may argue that the augmented samples of **GEAD** are not from the real distribution. However, the purpose of exploring LC spaces is to find inconsistent logic between two models, instead of misclassified samples. Future work could develop more augmentation approaches. Thirdly, the adaption of **GEAD** to the sequential case is initial and limited to univariate models. Future work should further investigate more general adaption approaches.

7 Conclusion

This work proposes **GEAD**, a novel global explanation for DL-based anomaly detection. Through extensive experiments over two security applications, we demonstrate the superiority of **GEAD** compared with existing IID sampling-based and tree explanations. We showcase three usages of **GEAD** including Model vs. Expert, Model vs. Model, and Model + Expert. Through **GEAD**, we validate valuable knowledge in DL-based anomaly detection systems, while also revealing gaps between anomaly rules and intended threats. In summary, this work demonstrates the promising potential of **GEAD** for opening the black box, alleviating the concerns of security professionals on the opacity of DL models. Furthermore, we underscore the necessity for continued efforts to bridge the divide between model and expert knowledge, offering a path forward for more reliable anomaly detection in security.

[5]To ensure the efficiency of and fairness, the hyperparameters of Trustee and KITree are tuned in the comparison experiments.

8 Acknowledgement

We are grateful to all the reviewers for their great effort and all the members from NMGroup, CNPT-Lab, and CSL. This work is supported by the National Key R&D Program of China under Grant 2022YFB3102902, the National Natural Science Foundation of China under Grant 62172251. This work is also partially supported by the National Research Foundation, Singapore, and the Cyber Security Agency under its National Cybersecurity R&D Programme (NCRP25-P04-TAICeN). Any opinions, findings and conclusions, or recommendations expressed in this material are those of the author(s) and do not reflect the views of National Research Foundation, Singapore and Cyber Security Agency of Singapore. Zhiliang Wang is the corresponding author of this paper.

References

[1] 2023. Snort IDS. https://www.snort.org/.
[2] Mohiuddin Ahmed, Abdun Naser Mahmood, and Jiankun Hu. 2016. A survey of network anomaly detection techniques. *Journal of Network and Computer Applications* 60 (2016), 19–31.
[3] Giuseppina Andresini, Feargus Pendlebury, Fabio Pierazzi, Corrado Loglisci, Annalisa Appice, and Lorenzo Cavallaro. 2021. INSOMNIA: Towards Concept-Drift Robustness in Network Intrusion Detection. In *Proceedings of the 2013 ACM Workshop on Artificial Intelligence and Security (AISec)*. ACM, 111–122.
[4] Liat Antwarg, Ronnie Mindlin Miller, Bracha Shapira, and Lior Rokach. 2021. Explaining anomalies detected by autoencoders using Shapley Additive Explanations. *Expert systems with applications* 186 (2021), 115736.
[5] Daniel W Apley and Jingyu Zhu. 2020. Visualizing the effects of predictor variables in black box supervised learning models. *Journal of the Royal Statistical Society Series B: Statistical Methodology* 82, 4 (2020), 1059–1086.
[6] Alberto Barbado, Óscar Corcho, and Richard Benjamins. 2022. Rule extraction in unsupervised anomaly detection for model explainability: Application to OneClass SVM. *Expert Systems with Applications* 189 (2022), 116100.
[7] Benjamin Bowman, Craig Laprade, Yuede Ji, and H Howie Huang. 2020. Detecting Lateral Movement in Enterprise Computer Networks with Unsupervised Graph AI. In *23rd International Symposium on Research in Attacks, Intrusions and Defenses (RAID)*. 257–268.
[8] Rich Caruana, Yin Lou, Johannes Gehrke, Paul Koch, Marc Sturm, and Noemie Elhadad. 2015. Intelligible models for healthcare: Predicting pneumonia risk and hospital 30-day readmission. In *Proceedings of the 21th ACM SIGKDD international conference on knowledge discovery and data mining*. 1721–1730.
[9] Davide Castelvecchi. 2016. Can we open the black box of AI? *Nature News* 538, 7623 (2016), 20.
[10] Varun Chandola, Arindam Banerjee, and Vipin Kumar. 2009. Anomaly detection: A survey. *ACM computing surveys (CSUR)* 41, 3 (2009), 1–58.
[11] Chun-Hao Chang, Elliot Creager, Anna Goldenberg, and David Duvenaud. 2019. Explaining Image Classifiers by Counterfactual Generation. In *ICLR*. OpenReview.net.
[12] Baijun Cheng, Shengming Zhao, Kailong Wang, Meizhen Wang, Guangdong Bai, Ruitao Feng, Yao Guo, Lei Ma, and Haoyu Wang. 2024. Beyond Fidelity: Explaining Vulnerability Localization of Learning-based Detectors. *arXiv preprint arXiv:2401.02686* (2024).
[13] Piotr Dabkowski and Yarin Gal. 2017. Real time image saliency for black box classifiers. *Advances in neural information processing systems* 30 (2017).
[14] Marius Drăgoi, Elena Burceanu, Emanuela Haller, Andrei Manolache, and Florin Brad. 2022. AnoShift: A Distribution Shift Benchmark for Unsupervised Anomaly Detection. *Neural Information Processing Systems NeurIPS, Datasets and Benchmarks Track* (2022).
[15] Min Du, Zhi Chen, Chang Liu, Rajvardhan Oak, and Dawn Song. 2019. Lifelong anomaly detection through unlearning. In *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. ACM, 1283–1297.
[16] Min Du, Feifei Li, Guineng Zheng, and Vivek Srikumar. 2017. Deeplog: Anomaly detection and diagnosis from system logs through deep learning. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. ACM, 1285–1298.

[17] Gints Engelen, Vera Rimmer, and Wouter Joosen. 2021. Troubleshooting an intrusion detection dataset: the CICIDS2017 case study. In *2021 IEEE Security and Privacy Workshops (SPW)*. IEEE, 7–12.

[18] Ming Fan, Wenying Wei, Xiaofei Xie, Yang Liu, Xiaohong Guan, and Ting Liu. 2020. Can We Trust Your Explanations? Sanity Checks for Interpreters in Android Malware Analysis. *IEEE Transactions on Information Forensics and Security* 16 (2020), 838–853.

[19] Ruth Fong, Mandela Patrick, and Andrea Vedaldi. 2019. Understanding Deep Networks via Extremal Perturbations and Smooth Masks. In *ICCV*. IEEE, 2950–2958.

[20] Ruth C Fong and Andrea Vedaldi. 2017. Interpretable explanations of black boxes by meaningful perturbation. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*. 3429–3437.

[21] Jerome H Friedman. 2001. Greedy function approximation: a gradient boosting machine. *Annals of statistics* (2001), 1189–1232.

[22] Prasanta Gogoi, DK Bhattacharyya, Bhogeswar Borah, and Jugal K Kalita. 2014. MLH-IDS: a multi-level hybrid intrusion detection method. *Comput. J.* 57, 4 (2014), 602–623.

[23] Akul Goyal, Xueyuan Han, Gang Wang, and Adam Bates. 2023. Sometimes, You Aren't What You Do: Mimicry Attacks against Provenance Graph Host Intrusion Detection Systems. In *30th ISOC Network and Distributed System Security Symposium (NDSS'23), San Diego, CA, USA*.

[24] Riccardo Guidotti, Anna Monreale, Salvatore Ruggieri, Franco Turini, Fosca Giannotti, and Dino Pedreschi. 2018. A survey of methods for explaining black box models. *ACM computing surveys (CSUR)* 51, 5 (2018), 1–42.

[25] Chun Guo, Yuan Ping, Nian Liu, and Shou-Shan Luo. 2016. A two-level hybrid approach for intrusion detection. *Neurocomputing* 214 (2016), 391–400.

[26] Wenbo Guo, Dongliang Mu, Jun Xu, Purui Su, Gang Wang, and Xinyu Xing. 2018. Lemna: Explaining deep learning based security applications. In *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. 364–379.

[27] Dongqi Han, Zhiliang Wang, Wenqi Chen, Kai Wang, Rui Yu, Su Wang, Han Zhang, Zhihua Wang, Minghui Jin, Jiahai Yang, et al. 2023. Anomaly Detection in the Open World: Normality Shift Detection, Explanation, and Adaptation. In *30th Annual Network and Distributed System Security Symposium (NDSS)*.

[28] Dongqi Han, Zhiliang Wang, Wenqi Chen, Ying Zhong, Su Wang, Han Zhang, Jiahai Yang, Xingang Shi, and Xia Yin. 2021. DeepAID: Interpreting and Improving Deep Learning-Based Anomaly Detection in Security Applications. In *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. ACM, 3197–3217.

[29] Congjie He, Meng Ma, and Ping Wang. 2020. Extract interpretability-accuracy balanced rules from artificial neural networks: A review. *Neurocomputing* 387 (2020), 346–358.

[30] Mark Ibrahim, Melissa Louie, Ceena Modarres, and John Paisley. 2019. Global explanations of neural networks: Mapping the landscape of predictions. In *Proceedings of the 2019 AAAI/ACM Conference on AI, Ethics, and Society*. 279–287.

[31] Alan Inglis, Andrew Parnell, and Catherine B Hurley. 2022. Visualizing variable importance and variable interaction effects in machine learning models. *Journal of Computational and Graphical Statistics* 31, 3 (2022), 766–778.

[32] Arthur S Jacobs, Roman Beltiukov, Walter Willinger, Ronaldo A Ferreira, Arpit Gupta, and Lisandro Z Granville. 2022. Ai/ml for network security: The emperor has no clothes. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*. 1537–1551.

[33] Jacob Kauffmann, Klaus-Robert Müller, and Grégoire Montavon. 2020. Towards explaining anomalies: a deep Taylor decomposition of one-class models. *Pattern Recognition* 101 (2020), 107198.

[34] Gisung Kim, Seungmin Lee, and Sehun Kim. 2014. A novel hybrid intrusion detection method integrating anomaly detection with misuse detection. *Expert Systems with Applications* 41, 4 (2014), 1690–1700.

[35] Isaiah J King and H Howie Huang. 2022. Euler: Detecting network lateral movement via scalable temporal link prediction. In *Network and Distributed Systems Security (NDSS) Symposium*.

[36] Martin Kopp, Tomáš Pevnỳ, and Martin Holeňa. 2020. Anomaly explanation with random forests. *Expert Systems with Applications* 149 (2020), 113187.

[37] Aditya Kuppa and Nhien-An Le-Khac. 2021. Adversarial xai methods in cybersecurity. *IEEE transactions on information forensics and security* 16 (2021), 4924–4938.

[38] Ruoyu Li, Qing Li, Yu Zhang, Dan Zhao, Yong Jiang, and Yong Yang. 2024. Interpreting Unsupervised Anomaly Detection in Security via Rule Extraction. *Advances in Neural Information Processing Systems* 36 (2024).

[39] Lisa Liu, Gints Engelen, Timothy Lynar, Daryl Essam, and Wouter Joosen. 2022. Error prevalence in nids datasets: A case study on cic-ids-2017 and cse-cic-ids-2018. In *2022 IEEE Conference on Communications and Network Security (CNS)*. IEEE, 254–262.

[40] Ninghao Liu, Donghwa Shin, and Xia Hu. 2018. Contextual outlier interpretation. In *Proceedings of the 27th International Joint Conference on Artificial Intelligence (IJCAI)*. 2461–2467.

[41] Wei-Yin Loh. 2011. Classification and regression trees. *Wiley interdisciplinary reviews: data mining and knowledge discovery* 1, 1 (2011), 14–23.

[42] Huimin Lu, Tian Wang, Xing Xu, and Ting Wang. 2021. Cognitive memory-guided autoencoder for effective intrusion detection in internet of things. *IEEE Transactions on Industrial Informatics* 18, 5 (2021), 3358–3366.

[43] Weibin Meng, Ying Liu, Yichen Zhu, Shenglin Zhang, Dan Pei, Yuqing Liu, Yihao Chen, Ruizhi Zhang, Shimin Tao, Pei Sun, and Rong Zhou. 2019. LogAnomaly: Unsupervised Detection of Sequential and Quantitative Anomalies in Unstructured Logs. In *Proceedings of the Twenty-Eighth International Joint Conference on Artificial Intelligence (IJCAI)*. 4739–4745.

[44] Zili Meng, Minhu Wang, Jiasong Bai, Mingwei Xu, Hongzi Mao, and Hongxin Hu. 2020. Interpreting Deep Learning-Based Networking Systems. In *Proceedings of the Annual conference of the ACM Special Interest Group on Data Communication on the applications, technologies, architectures, and protocols for computer communication (SIGCOMM)*. 154–171.

[45] Yisroel Mirsky, Tomer Doitshman, Yuval Elovici, and Asaf Shabtai. 2018. Kitsune: an ensemble of autoencoders for online network intrusion detection. *25th Annual Network and Distributed System Security Symposium (NDSS)*.

[46] Chirag N Modi and Dhiren Patel. 2013. A novel hybrid-network intrusion detection system (H-NIDS) in cloud computing. In *2013 IEEE Symposium on Computational Intelligence in Cyber Security (CICS)*. IEEE, 23–30.

[47] Grégoire Montavon, Sebastian Lapuschkin, Alexander Binder, Wojciech Samek, and Klaus-Robert Müller. 2017. Explaining nonlinear classification decisions with deep Taylor decomposition. *Pattern Recognition* 65 (2017), 211–222.

[48] Andrea Morichetta, Pedro Casas, and Marco Mellia. 2019. EXPLAIN-IT: Towards Explainable AI for Unsupervised Network Traffic Analysis. In *Proceedings of the 3rd ACM CoNEXT Workshop on Big DAta, Machine Learning and Artificial Intelligence for Data Communication Networks*. 22–28.

[49] Luis Muñoz-González, Battista Biggio, Ambra Demontis, Andrea Paudice, Vasin Wongrassamee, Emil C Lupu, and Fabio Roli. 2017. Towards poisoning of deep learning algorithms with back-gradient optimization. In *Proceedings of the 10th ACM workshop on artificial intelligence and security*. 27–38.

[50] Gregory Plumb, Denali Molitor, and Ameet S Talwalkar. 2018. Model agnostic supervised local explanations. *Advances in neural information processing systems* 31 (2018).

[51] Marco Tulio Ribeiro, Sameer Singh, and Carlos Guestrin. 2016. "Why should I trust you?" Explaining the predictions of any classifier. In *Proceedings of the 22nd ACM SIGKDD international conference on knowledge discovery and data mining (KDD)*. 1135–1144.

[52] Emad W Saad and Donald C Wunsch II. 2007. Neural network explanation using inversion. *Neural networks* 20, 1 (2007), 78–93.

[53] Iman Sharafaldin, Arash Habibi Lashkari, and Ali A Ghorbani. 2018. Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization.. In *Proceedings of the 2nd international conference on information systems security and privacy (ICISSP)*. 108–116.

[54] Iman Sharafaldin, Arash Habibi Lashkari, and Ali A Ghorbani. 2018. Toward generating a new intrusion detection dataset and intrusion traffic characterization. *ICISSp* 1 (2018), 108–116.

[55] Avanti Shrikumar, Peyton Greenside, and Anshul Kundaje. 2017. Learning important features through propagating activation differences. In *International Conference on Machine Learning(ICML)*. 3145–3153.

[56] Daniel Smilkov, Nikhil Thorat, Been Kim, Fernanda Viégas, and Martin Wattenberg. 2017. Smoothgrad: removing noise by adding noise. *arXiv preprint arXiv:1706.03825* (2017).

[57] Fei Song, Yanlei Diao, Jesse Read, Arnaud Stiegler, and Albert Bifet. 2018. EXAD: A system for explainable anomaly detection on big data traces. In *2018 IEEE International Conference on Data Mining Workshops (ICDMW)*. IEEE, 1435–1440.

[58] Jungsuk Song, Hiroki Takakura, Yasuo Okabe, Masashi Eto, Daisuke Inoue, and Koji Nakao. 2011. Statistical analysis of honeypot data and building of Kyoto 2006+ dataset for NIDS evaluation. In *Proceedings of the first workshop on building analysis datasets and gathering experience returns for security*. 29–36.

[59] Mukund Sundararajan, Ankur Taly, and Qiqi Yan. 2017. Axiomatic attribution for deep networks. In *ICML (Proceedings of Machine Learning Research, Vol. 70)*. PMLR, 3319–3328.

[60] Ruming Tang, Zheng Yang, Zeyan Li, Weibin Meng, Haixin Wang, Qi Li, Yongqian Sun, Dan Pei, Tao Wei, Yanfei Xu, et al. 2020. Zerowall: Detecting zero-day web attacks through encoder-decoder recurrent neural networks. In *39th IEEE Conference on Computer Communications (INFOCOM)*. IEEE, 2479–2488.

[61] Ruoying Wang, Kexin Nie, Tie Wang, Yang Yang, and Bo Long. 2020. Deep Learning for Anomaly Detection. In *Proceedings of the 13th International Conference on Web Search and Data Mining (WSDM)*. 894–896.

[62] Su Wang, Zhiliang Wang, Tao Zhou, Hongbin Sun, Xia Yin, Dongqi Han, Han Zhang, Xingang Shi, and Jiahai Yang. 2022. Threatrace: Detecting and tracing host-based threats in node level through provenance graph learning. *IEEE Transactions on Information Forensics and Security* 17 (2022), 3972–3987.

[63] Alexander Warnecke, Daniel Arp, Christian Wressnegger, and Konrad Rieck. 2020. Evaluating explanation methods for deep learning in security. In *2020 IEEE European Symposium on Security and Privacy (EuroS&P)*. IEEE, 158–174.

[64] Feng Wei, Hongda Li, Ziming Zhao, and Hongxin Hu. 2023. XNIDS: Explaining Deep Learning-based Network Intrusion Detection Systems for Active Intrusion Responses. In *32nd USENIX Security Symposium (USENIX Security 23), Anaheim, CA, USA.*

[65] Mike Wu, Michael C Hughes, Sonali Parbhoo, Maurizio Zazzi, Volker Roth, and Finale Doshi-Velez. 2018. Beyond Sparsity: Tree Regularization of Deep Models for Interpretability. In *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI).*

[66] Haowen Xu, Wenxiao Chen, Nengwen Zhao, Zeyan Li, Jiahao Bu, Zhihan Li, Ying Liu, Youjian Zhao, Dan Pei, Yang Feng, et al. 2018. Unsupervised anomaly detection via variational auto-encoder for seasonal kpis in web applications. In *Proceedings of the 2018 World Wide Web Conference (WWW).* 187–196.

[67] Hongzuo Xu, Yijie Wang, Songlei Jian, Zhenyu Huang, Yongjun Wang, Ning Liu, and Fei Li. 2021. Beyond outlier detection: Outlier interpretation by attention-guided triplet deviation network. In *Proceedings of the Web Conference 2021.* 1328–1339.

[68] Wei Xu, Ling Huang, Armando Fox, David Patterson, and Michael I Jordan. 2009. Detecting large-scale system problems by mining console logs. In *Proceedings of the ACM SIGOPS 22nd symposium on Operating systems principles (SOSP).* 117–132.

[69] Fan Yang, Jiacen Xu, Chunlin Xiong, Zhou Li, and Kehuan Zhang. 2023. {PROGRAPHER}: An Anomaly Detection System based on Provenance Graph Embedding. In *32nd USENIX Security Symposium (USENIX Security 23).* 4355–4372.

[70] Limin Yang, Wenbo Guo, Qingying Hao, Arridhana Ciptadi, Ali Ahmadzadeh, Xinyu Xing, and Gang Wang. 2021. CADE: Detecting and Explaining Concept Drift Samples for Security Applications. In *30th USENIX Security Symposium (USENIX Security).*

[71] Véronne Yepmo, Grégory Smits, and Olivier Pivert. 2022. Anomaly explanation: A review. *Data & Knowledge Engineering* 137 (2022), 101946.

[72] Sultan Zavrak and Murat İskefiyeli. 2020. Anomaly-based intrusion detection from network flow features using variational autoencoder. *IEEE Access* 8 (2020), 108346–108358.

[73] Houssam Zenati, Manon Romain, Chuan-Sheng Foo, Bruno Lecouat, and Vijay Chandrasekhar. 2018. Adversarially learned anomaly detection. In *2018 IEEE International Conference on Data Mining (ICDM).* IEEE, 727–736.

[74] Jun Zengy, Xiang Wang, Jiahao Liu, Yinfang Chen, Zhenkai Liang, Tat-Seng Chua, and Zheng Leong Chua. 2022. Shadewatcher: Recommendation-guided cyber threat analysis using system audit records. In *2022 IEEE Symposium on Security and Privacy (S&P).* IEEE, 489–506.

[75] Jiong Zhang and Mohammad Zulkernine. 2006. A hybrid network intrusion detection technique using random forests. In *First International Conference on Availability, Reliability and Security (ARES'06).* IEEE, 8–pp.

[76] Chong Zhou and Randy C Paffenroth. 2017. Anomaly detection with robust deep autoencoders. In *Proceedings of the 23rd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD).* 665–674.

[77] Luisa M. Zintgraf, Taco S. Cohen, Tameem Adel, and Max Welling. 2017. Visualizing Deep Neural Network Decisions: Prediction Difference Analysis. In *ICLR.* OpenReview.net.

A GEAD Design for Sequential Model

As mentioned in §3.2, we need some adaptations for **GEAD** used for the sequential model. Except for the differences introduced in the main body of the paper, we introduce the detail modification on **GEAD** w.r.t. LC Augmentation. In this case, Eq. (1) is changed to:

$$[\mathrm{X}_i^-]_j = [\mathrm{X}_i^-]_{j-1} + \beta \cdot \frac{\partial \mathcal{F}([\mathrm{X}_i^-]_{j-1})}{\partial [\mathrm{X}_i^-]_{j-1}}, \\ [\mathrm{X}_i^+]_j = [\mathrm{X}_i^+]_{j-1} - \beta \cdot \frac{\partial \mathcal{F}([\mathrm{X}_i^+]_{j-1})}{\partial [\mathrm{X}_i^+]_{j-1}}. \tag{2}$$

Because the output of the prediction-based model is the normal score, we need to reverse the direction of the gradient computation. At the same time, it should be noted that in our univariate model, since the value at each time point is discrete, we need to discretize the augmented data eventually.

B Experimental Supplement

B.1 Hyperparameter Sensitivity

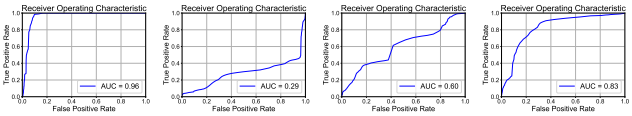
Here we evaluate the sensitivity of two hyperparameters N and β in **GEAD**. The other parameters need to be determined using the search method introduced in §4.1. We sample both parameters from a reasonable range, and the variation of the explanation quality is listed in Table 5. The results demonstrate the insensitivity of these two parameters. Note that the insensitivity must be held within a (relatively large) range.

Table 5: Sensitivity test of hyper-parameters in GEAD.

Parameters	Default	Testing Range	Pos. Consistency	Neg. Credibility
β	0.1	[0.01, 1.]	[0.909, 0.923]	[0.987, 0.987]
N	20	[5, 50]	[0.923, 0.934]	[0.980, 0.987]

B.2 Model Performance under Concept drift

Here are the ROCs used in §4.3.2 (Usage 2).We use the Kyoto dataset as it consists of a long period. The model is trained with normal traffic in 2007 and tested with traffic in 2014. As shown in Fig. 10, there is a significant drift from 2007 to 2014, and a significant degradation in model performance is witnessed.



(a) Test@2007 **(b) Test@2014** **(c) Retraining** **(d) OWAD**

Figure 10: ROC of models before and after concept drift (Kyoto dataset).

B.3 Verification of Insight 4

We also empirically justify the second part of Insight 4 through two experiments: one is to flip the features close to the leaves and observe how many abnormal samples are flipped as normal; the other is to leverage the local explanation DeepAID [28] to verify the important features in certain decisions. We conduct the experiments on all rules of the model trained from CICIDS2017 dataset. The results are shown in Table 6. The larger the two metrics are, the more important the feature is. It can be seen that the feature closer to the leaf is important for the particular decision.

Table 6: Evaluation of feature importance within certain rules

Distance from the Leaf	Feature Flipping # abnormal to # normal	DeepAID Gradient
1	20	4.50
2	1	2.20
3	1	0.46
4	0	-0.04
5	-5	0

C GEAD Trees and Rules

We provide the rule tree plots used in the evaluation in this paper. Due to space limit, this appendix is available at https://github.com/dongtsi/GEAD/blob/main/doc/tree_appendix.pdf.